
Characterizing Reinforcement Learning (RL) Environments by their Exploration Requirement

Vladislav Antipin

Sorbonne Université

vladislav.antipin@etu.sorbonne-universite.fr

Milos Oundjian

Sorbonne Université

milos.oundjian@etu.sorbonne-universite.fr

Abstract

The objective of offline RL is to learn a good policy for an environment based only on a fixed dataset of transitions (replay buffer) in that environment. The exploitation-exploration tradeoff is present in the composition of this fixed dataset. The goal of this project is to evaluate how the makeup of the replay buffer affects the performance of offline learning on different environments. We first look at a replay buffer of mixtures of different proportions of uniformly drawn transitions from the environment and transitions from a policy of certain performance. We then also at replay buffers drawn from policies of certain performances with different amounts of noise added. We also create a new way of generating noised replay buffers that we call “branching noise”. Our experiments show that uniform exploration is practically always beneficial for learning all environments, whereas action noise generally results in poorer performance and has an optimum noise level for some environments.

1 Introduction

RL algorithms are traditionally online, meaning that the agent continuously interacts with the environment during the learning process. This has the advantage of allowing the agent to carry out exploration by interacting with the environment. However, often collecting new interactions can be costly, slow or impractical.

Offline RL aims to address this limitation by learning a policy exclusively from a fixed dataset of previously collected transitions, without requiring additional interaction with the environment.

Transitions are stored in a replay buffer, a dataset containing tuples of states, actions, rewards, and next states collected from a single or multiple policies. The quality and composition of this replay buffer play a central role in the performance of offline RL algorithms.

One important challenge is the exploration–exploitation tradeoff. In the offline setting, this tradeoff is reflected in the diversity of the replay buffer (Lambert et al., 2022). At one extreme, the dataset may contain transitions generated by a highly optimized policy, leading to strong exploitation but limited coverage of the state-action space. At the other extreme, the dataset may contain mostly random actions, providing broad exploration but lower-quality trajectories. Determining the balance between these two extremes is essential for effective offline learning.

Another major challenge is out-of-distribution (OOD) error. Because offline algorithms cannot interact with the environment during training, they must estimate the value of actions using only the available dataset. When the learned policy considers actions or state-action pairs that are poorly represented in the replay buffer, the value estimates can become inaccurate due to extrapolation beyond the training

distribution. These errors may accumulate during training and significantly degrade performance. Consequently, the composition of the replay buffer may play a role in reducing OOD errors by improving coverage of the state-action space.

This project studies the impact of exploration in offline RL datasets. More specifically, we investigate two research questions: **(1) how can environments be characterized according to the level of exploration required for effective offline learning**, and **(2) how do different exploration strategies compare across environments**. By analyzing the performance of offline RL algorithms on datasets with different exploration properties, we aim to better understand the relationship between replay buffer composition and learning performance.

Our work shows that **(1) all environments generally benefit from stronger exploration, with environment-specific exceptions arising from OOD errors or loss of learning signal**, and **(2) uniform exploration generally outperforms noise-based strategies, while the choice between action noise and branching noise is environment-dependent**.

2 Related Works

The challenges associated with offline RL have been widely studied in the literature, particularly issues related to distributional shift and out-of-distribution errors. Existing approaches can broadly be divided into two categories. The first focuses on algorithmic methods designed to improve the stability and robustness of learning from fixed datasets. The second focuses on dataset construction, with the goal of generating datasets whose properties better support effective offline learning.

Several algorithms have been proposed specifically to address the challenges of offline RL. TD3+BC (Fujimoto & Gu, 2021) extends TD3 by adding a behavior cloning objective that constrains the learned policy toward actions present in the dataset. Implicit Q-Learning (IQL) (Kostrikov et al., 2021) avoids querying unseen actions during policy improvement, thereby reducing extrapolation error. Conservative Q-Learning (CQL) (Kumar et al., 2020) instead penalizes overestimated values for out-of-distribution actions in order to learn more conservative value estimates. These approaches all aim to mitigate the instability caused by distributional shift between the learned policy and the offline dataset.

Some works instead focus on the construction and quality of replay buffers for offline RL. In particular (Lambert et al., 2022) studies the problem of collecting informative datasets independently from the downstream task. Their approach first generates replay buffers using different exploration strategies and only later assigns rewards depending on the task to be solved. This decouples data collection from policy learning and allows the same dataset to be reused across multiple tasks. Using this framework, they evaluate how different exploration methods affect the quality and coverage of the collected data for offline RL. Their work highlights the importance of replay buffer composition and motivates studying how exploration influences offline learning performance.

3 Background

In this section we cover the background in RL required to understand this project.

3.1 MDPs

A **Markov Decision Process** (MDP) is defined by a tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$ (Sutton & Barto, 2018).

- $\mathcal{S}$ is the state space of the problem, the possible states the agent can be in.
- $\mathcal{A}$ is the action space of the problem, the set of possible actions the agent can do.
- $T : \mathcal{S} \times \mathcal{A} \rightarrow \Pi(\mathcal{S})$ is the transition function, it maps state action pairs (s, a) to a probability distribution of the next possible states. If the environment is deterministic then $\Pi(\mathcal{S})$ is the dirac probability distribution.

- $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is reward function. It maps state action pairs (s, a) to a real valued reward.
- γ is the discount factor. This is used for evaluation of cumulative reward to determine the importance of future rewards compared to immediate rewards.

A deterministic policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ is a mapping from state to action (policies can also be stochastic). The goal of RL is to learn the policy that maximizes the expected cumulative reward on an environment given by

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right].$$

Transitions are the dataset building blocks of RL algorithms. A transition is a tuple $(s, a, r, s') \in \mathcal{S} \times \mathcal{A} \times \mathbb{R} \times \mathcal{S}$. Transition may contain a flag variable for if the environment terminated (terminated or truncated depending on whether the terminal state was from the episode ending due to the episode time limit) A transition is collected on an environment by an agent following a policy. Transitions are stored in a replay buffer.

3.2 RL Algorithms

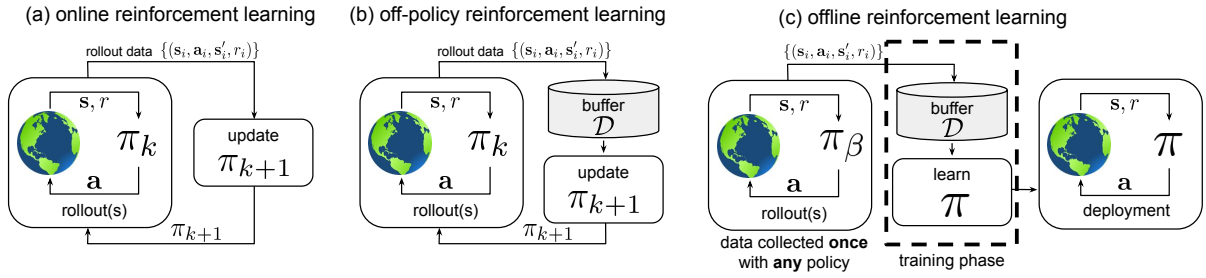


Figure 3.1: The three types of RL algorithms. (a) shows online RL. Note the absence of a replay buffer. (b) shows off-policy RL. (c) Shows offline RL. It is important to note that once the buffer is collected no new transitions are collected during training for offline learning. Source: (Levine et al., 2020).

There are two separate categories in RL algorithm implementations:

1. **On-policy** vs **Off-policy**
2. **Online** vs **Offline**.

On-policy RL algorithms can only learn from data generated by their current policy. More precisely, the transitions used to update a policy π_k into the next policy π_{k+1} must be collected using π_k . These methods are generally more stable because the training distribution closely matches the policy being optimized. However, they are typically sample inefficient since previously collected transitions cannot easily be reused once the policy changes.

Off-policy algorithms, on the other hand, can learn from transitions generated by a different policy than the one currently being optimized. This allows the reuse of past experience through the use of a replay buffer, improving sample efficiency. The replay buffer may contain transitions collected using older versions of the policy, exploratory policies or any other way.

In **online RL**, the agent continuously interacts with the environment during training. New transitions are collected while the policy is being updated. Online can be either on-policy or off-policy depending on how the collected transitions are used during learning.

In **offline RL**, the agent can no longer interact with the environment during training. Instead, a fixed dataset of transitions is collected beforehand. This dataset, stored in a replay buffer, remains fixed throughout training.

An on-policy and offline algorithm is contradictory. Figure 3.1 illustrates the 3 compatible combinations.

Continuous environments, as opposed to discrete ones, have the action and state space be continuous rather than discrete. In such settings, standard tabular Q-learning methods are not directly applicable.

To address this problem, actor-critic methods based on Q-learning principles, such as those proposed by (Konda & Tsitsiklis, 1999), provide a better solution: a critic network approximates the Q-function, while an actor network learns a deterministic policy that directly outputs continuous actions.

Twin Delayed Deep Deterministic Policy Gradient (TD3) (Fujimoto et al., 2018). TD3 is an off-policy actor-critic algorithm designed for continuous action spaces and based on Deep Deterministic Policy Gradient (DDPG). A major issue in Q-learning-based actor-critic methods is overestimation bias in the critic estimates. TD3 mitigates this issue through clipped double Q-learning, where two critic networks are trained simultaneously and the minimum of their predictions is used when computing target values. TD3 also introduces delayed policy updates and target policy smoothing, which further improve training stability.

3.3 Issues Specific to Offline RL

One of the main challenges in offline RL is **distributional shift** (Wang et al., 2024). Since offline RL algorithms train on a fixed replay buffer, they cannot collect additional transitions to correct inaccurate estimates during training.

This becomes particularly problematic when using function approximation, such as neural-network-based Q-functions. During policy improvement, the learned policy may select actions that are absent or poorly represented in the replay buffer. In these regions of the state-action space, the critic must extrapolate from limited data, which can lead to severe overestimation errors. Because no new transitions are collected in offline RL, these incorrect value estimates cannot be corrected through further interaction with the environment.

This issue is especially important in Q-learning-based methods, where the policy is optimized to maximize the estimated Q-values. If the critic incorrectly assigns a high value to out-of-distribution actions, the learned policy may exploit these errors and diverge from the behavior policy that generated the dataset, leading to poor performance.

Many offline reinforcement learning methods therefore attempt to constrain the learned policy to remain close to the behavior policy or regularize the critic to avoid unrealistic Q-value estimates.

One method specifically designed to address this issue is **Implicit Q-Learning (IQL)** (Kostrikov et al., 2021). Unlike regular RL algorithms, IQL does not evaluate actions outside the replay buffer during training. By avoiding queries of unseen actions, IQL significantly reduces the impact of distributional shift and overestimation errors in offline reinforcement learning. However the downside is that the policy learnt cannot deviate significantly from the dataset, meaning that if the replay buffer is collected from an inferior policy, IQL may not be able to generate a better policy.

4 Methods

In this section we describe the specifics of the different methods used to generate the replay buffers.

4.1 Replay Buffer Collection

Uniform Sampling-Policy Mix: The first method generates replay buffers by combining uniformly sampled transitions with transitions collected by following a pre-learned policy. The goal of this method is to control the balance between exploration and exploitation in the dataset.

The first component, uniform sampling, generates exploratory transitions by initializing the environment in randomly sampled states and selecting actions uniformly at random from the action space. This produces transitions that cover a broad portion of the state–action space.

The second component, policy sampling, generates exploitative transitions by running a pre-learned policy in the environment and collecting the resulting trajectories. These transitions are concentrated around behaviors that achieve high reward.

The final replay buffer is constructed by mixing these two sources of transitions. The proportion of uniformly sampled transitions determines the exploration level of the dataset.

The pseudocode for this method is shown in Listing A.1.

Action Noise: The second method generates replay buffers by running a pre-learned policy while adding truncated Gaussian noise characterized by its variance σ to its actions during execution. Unlike the previous method, exploration is introduced locally around the trajectories induced by the policy rather than through fully random transitions.

This method is similar to the exploration strategy commonly used in online reinforcement learning algorithms such as TD3.

The pseudocode for this method is shown in Listing A.2.

Branching Noise: The third method used to collect transitions is one that we came up with called “branching noise”.

As an episode is collected by executing the pre-learned policy in the environment, for each transition encountered along this trajectory, additional exploratory trajectories are generated by restarting from the corresponding state and executing a noisy version of the policy for d steps. This branching procedure is repeated k times for each state.

The noisy policy uses truncated Gaussian action noise in the same manner as the action noise method, with σ controlling the exploration level. The parameters d and k respectively determine the branch depth and the number of branches generated per state.

The objective of this method is to enrich the replay buffer with transitions located near the trajectories induced by the policy, while still exploring alternative behaviors in the surrounding state–action space.

The pseudocode for this method is shown in Listing A.3.

4.2 Offline Training

Once a replay buffer has been collected, training is performed entirely offline. This is done with TD3 for all three methods of replay buffer collection. The training is also done with IQL for the action noise and branching noise replay buffers.

At each training step, batches of transitions are sampled from the replay buffer and used to update the policy and value networks according to the chosen offline reinforcement learning algorithm. Since the dataset is fixed, the quality and diversity of the replay buffer play an important role in the final performance of the learned policy.

In our experiments, we evaluate how different replay buffer generation methods and exploration levels affect offline learning performance. For each replay buffer configuration (uniform, action, branching), we train an offline RL agent using the same training procedure and hyper-parameters across environments to ensure fair comparison between datasets.

Training performance is evaluated periodically by executing the learned policy in the environment without exploration noise and recording the average episodic return over several evaluations. This allows us to analyze the evolution of the performance of the learning throughout training.

All the code used for generating the replay buffers and running the RL algorithms is available on our github¹.

5 Experimental Setup

5.1 The environments

To evaluate our methods, we selected three continuous-control environments from Gymnasium (Towers et al., 2024). These environments were chosen because they present different exploration difficulties and control dynamics while remaining computationally lightweight, making them well suited for large-scale offline reinforcement learning experiments with multiple seeds.

1. **CartPoleContinuous-v1.** This environment is the continuous-action variant of **CartPole-v1**. The objective is to balance a pole on top of a cart moving along a one-dimensional track by applying continuous horizontal forces to the cart. Successful control requires maintaining stability while continuously correcting the pole's position and angle.
2. **MountainCarContinuous-v0.** This environment is the continuous-action variant of **MountainCar-v0**. The objective is to drive an underpowered car up a steep hill to reach a goal positioned at the top of the mountain. Since the car's engine is not strong enough to directly climb the slope, the agent must learn to build momentum by moving back and forth across the valley. Additionally, the environment penalizes large actions, creating a trade-off between exploration and energy efficiency.
3. **Pendulum-v1.** In this environment, the objective is to apply continuous torque to a pendulum in order to swing it upward and stabilize it in the upright position. The reward function favors states where the pendulum remains close to vertical while also penalizing large control inputs.

Figure 5.2 shows an illustration of what each environment looks like.

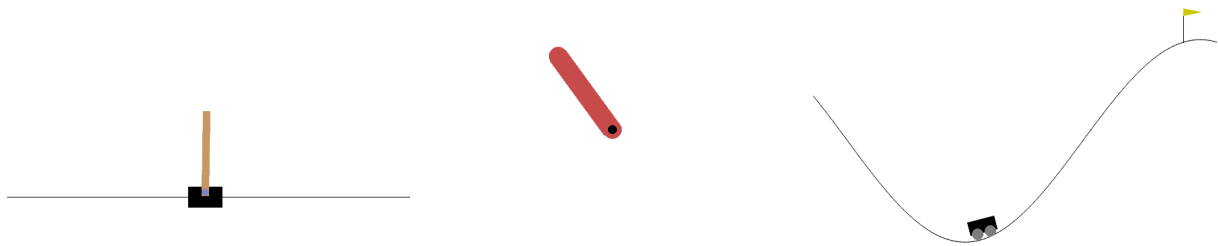


Figure 5.2: Illustrations of the three environments used in the experiments. From left to right: CartPoleContinuous-v1, Pendulum-v1, MountainCarContinuous-v0,

¹<https://github.com/milosoundjian/projet-mind>

	CartPoleContinuous-v1	Pendulum-v1	MountainCarContinuous-v0
State Space	Cart Position: $[-4.8, 4.8]$ Cart Velocity: $] -\infty, \infty[$ Pole Angle: $[-0.418, 0.418]$ Ang Velocity: $] -\infty, \infty[$	$\cos(\theta)$: $[-1, 1]$ $\sin(\theta)$: $[-1, 1]$ Ang Velocity: $[-8, 8]$	Car Position: $[-1.2, 0.6]$ Car Velocity: $[-0.07, 0.07]$
Action Space	Force: $[-1, 1]$	Torque: $[-2, 2]$	Force: $[-1, 1]$

Table 5.1: State and Action spaces of the different environments used in our experiments.

Table 5.1 shows the state and action spaces of the environments we are considering. Note that CartPole’s velocity and angular velocity are unbounded; for these dimensions we sampled from a standard normal distribution. As shown in Figure 5.4a and Figure A.7, this choice provided sufficient coverage of the regions relevant to the environment.

5.2 Algorithm Implementations and Hyperparameters

The initial implementations of our algorithms were carried out using the BBRL framework (Denoyer et al., 2021). BBRL provided a simple environment for experimenting with online and offline reinforcement learning methods, as well as for rapidly prototyping our replay buffer generation procedures and evaluation pipelines.

Eventually we transitioned to the D3RLpy library (Seno & Imai, 2022). D3RLpy offers implementations specifically designed for offline reinforcement learning and includes optimized, well-tested versions of several state-of-the-art algorithms, including IQL. Using D3RLpy simplified experimentation, improved reproducibility, and allowed us to focus more directly on analyzing the effects of replay buffer composition rather than maintaining low-level algorithmic implementations and hoping that our implementation was correct.

The two TD3 implementations differ slightly: our BBRL implementation does not use target policy noise, unlike D3RLpy. Consequently, for the same hyperparameters, D3RLpy converges faster across most environments. We therefore report D3RLpy results throughout, noting that BBRL results are largely consistent, as shown in Figure A.12.

Hyperparameters for TD3 were environment-specific adapted from Stable Baselines3 RL Zoo github repository², which provides well-tuned configurations validated across a wide range of environments, with the exception of the MLP architecture which we reduced from $[400, 300]$ to $[32, 16]$, as our environments are low-dimensional and the corresponding policies are relatively simple. The same hyperparameters were reused for IQL, as both algorithms share the same critic architecture and the TD3 hyperparameters yielded satisfactory performance in our experiments.

²<https://github.com/DLR-RM/rl-baselines3-zoo/blob/master/hyperparams/td3.yml>

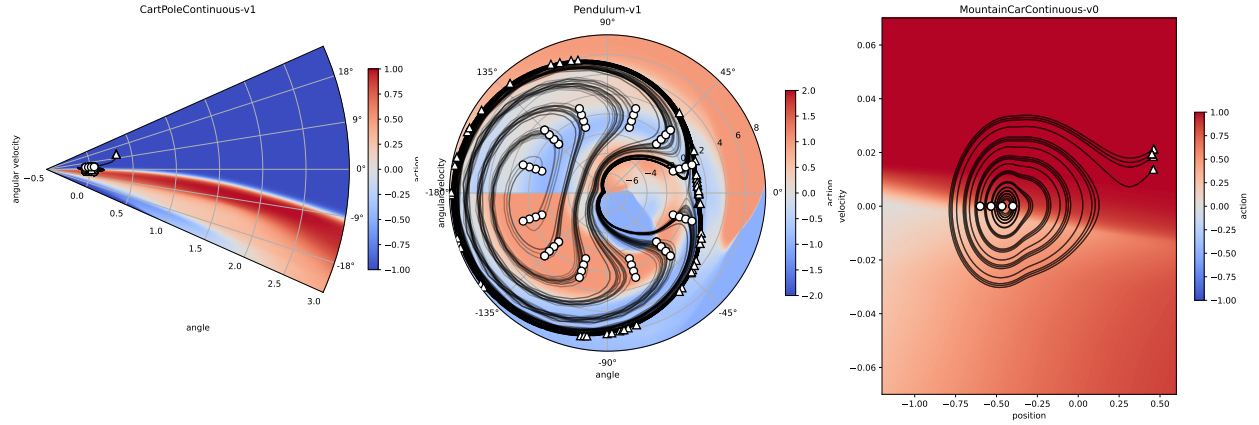


Figure 5.3: Online-learned policies retained for exploitation. Color encodes the action proposed by the policy at each point in the state space. For CartPole, a slice through state space is provided and other state variables are fixed at zero.. Black trajectories correspond to 4 complete episodes, the white circle indicates the start of an episode, and the triangle indicates the end. Retained policies yield an average reward of 500, -209, and 61 for CartPole, Pendulum, and MountainCar, respectively.

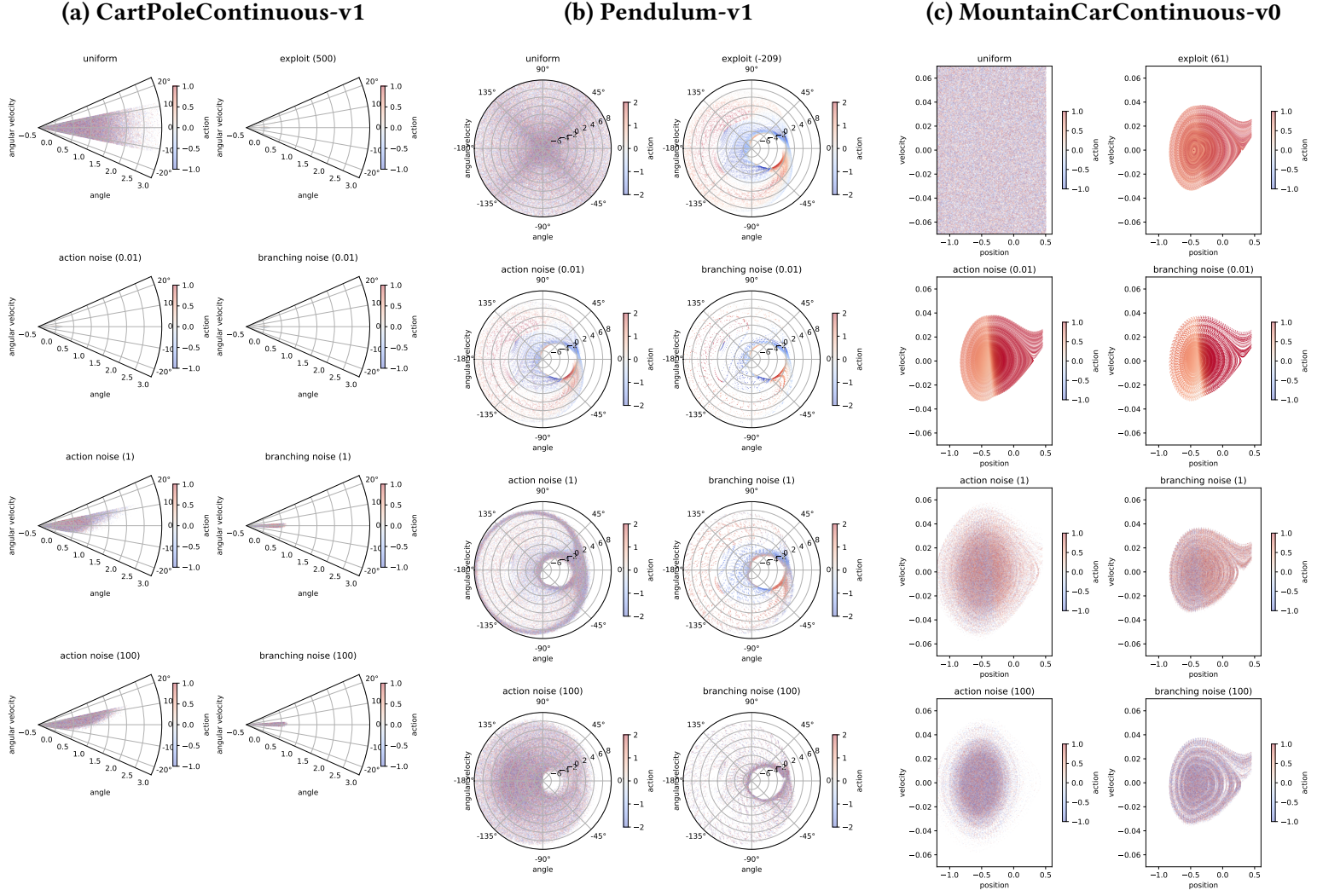


Figure 5.4:

Replay buffer visualizations in state-action space. **(a)** CartPole: slice through state space is provided and other state variables are fixed at zero. **(b)** Pendulum. **(c)** MountainCar. Exploitation policies have average reward of 500, -209, and 61 respectively. Truncated Gaussian noise’s standard deviation is shown in parentheses.

5.3 Result Collection Process

The general process for our experiments follows the following steps.

1. Learn policies through online reinforcement learning.

In the first stage, we train policies online using TD3 on each of the three Gymnasium environments. The goal is to obtain policies with optimal or near-optimal performance that can later be used to generate replay buffers for offline learning. During training, we also save intermediate policies at different stages of convergence. These partially trained policies achieve lower average evaluation rewards than the final policy and allow us to study how the quality of the data-collection policy influences offline learning performance. Example visualizations of suboptimal and optimal policies can be found in Figure A.7, Figure A.8 and Figure A.9. Preliminary results depicted in Figure A.10 and Figure A.11 suggest that a near-optimal exploitation policy leads to degraded offline learning performance for most exploration strategies in Pendulum and MountainCar. To ensure a fair comparison, we selected exploitation policies that enable effective offline learning, yielding average performances of 500, -209, and 61 for CartPole, Pendulum, and MountainCar respectively.

2. Generate replay buffers.

Using the policies obtained in the previous step, we generate replay buffers with 200,000 transitions, according to the collection methods described in the Methods section. This produces datasets with different exploration–exploitation characteristics.

3. Obtain learning curves through online learning.

Once a replay buffer has been generated, we train an offline RL agent exclusively on this fixed dataset, without any additional interaction with the environment. During training, we periodically evaluate the learned policy in the environment and record its performance over time. We collect the learning curves across multiple seeds for different replay buffers.

Examples of these learning curves are shown in Figure A.13 to Figure A.18.

4. Analyze performance evolution.

To compare the effects of different replay buffer compositions, we analyze how policy performance evolves during offline training. The plots presented in this paper show the average evaluation reward achieved after a given number of offline training steps for different exploration settings (noise levels or uniform-sampling proportions).

Since reinforcement learning training curves are typically noisy and highly variable, we apply smoothing to the recorded evaluation curves and average the performance over a small window of training steps around the selected point. This makes the overall learning trends easier to interpret and compare across experiments.

These are the curves shown in Figure 6.5, Figure 6.6 and Figure A.12.

In order to give evidence for explaining our results, we also created plots from the replay buffers and policies from the different replay buffer collection methods in each environments. This is shown in Figure 5.4.

6 Results

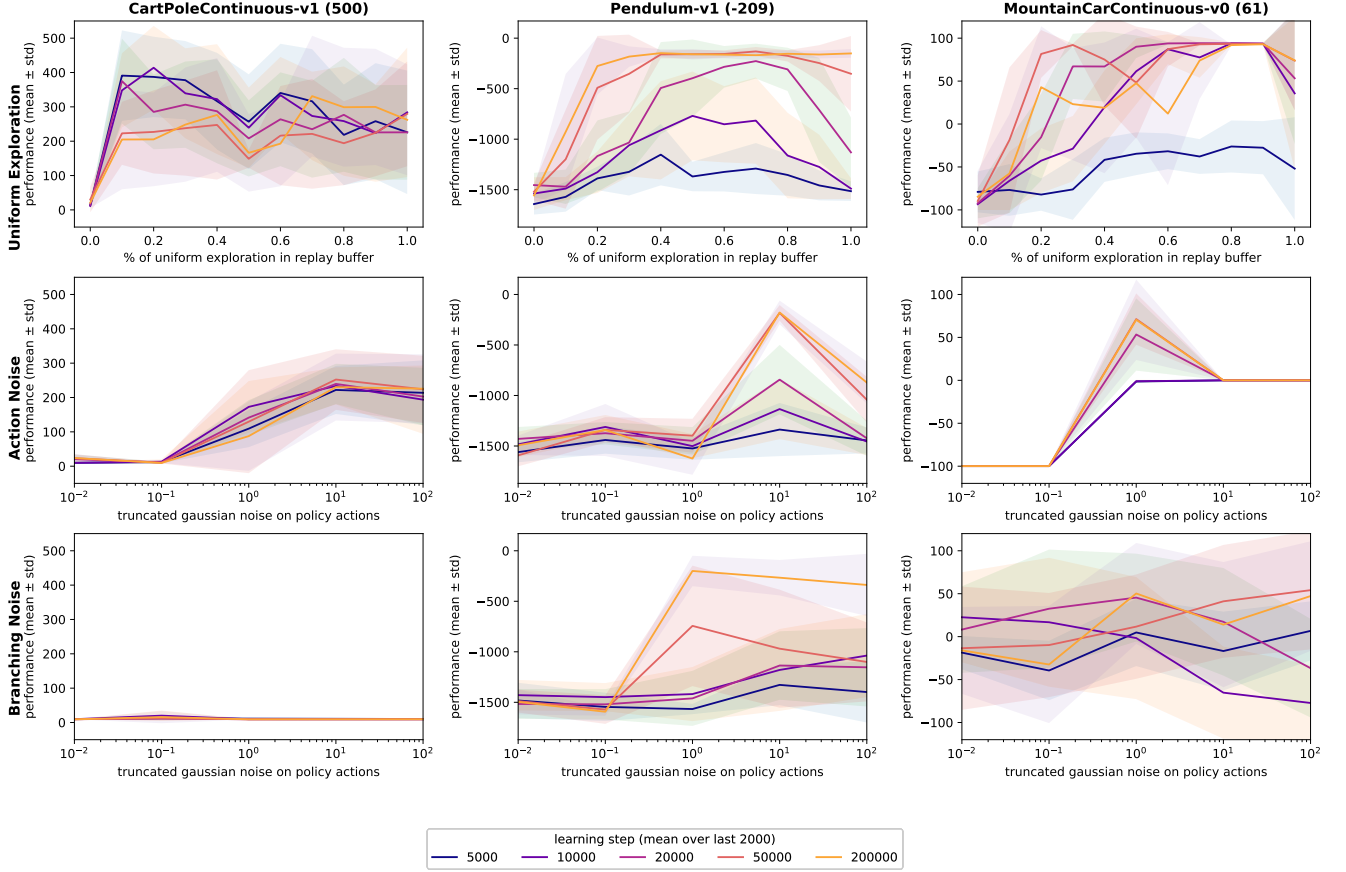


Figure 6.5: Evaluation performance during TD3 offline learning for varying replay buffer compositions. Three exploration strategies were used across three environments, and the resulting performances, averaged over the last 2000 learning steps, are shown as a function of replay buffer composition. Color depicts the learning step at which the evaluation performance was recorded. Performance was averaged over 10 seeds and 3 evaluations per seed every 1000 gradient steps. Shaded region corresponds to standard deviation. Average performance over 10 episodes of the exploitation policies used to obtain the corresponding replay buffers is indicated in parentheses. Y-axes are aligned so that each environment’s performance is depicted between the random baseline level and the maximum achievable reward. One step corresponds to one gradient update.

6.1 Uniform Exploration

As shown in the first row of Figure 6.5, varying the proportion of uniform exploration in the replay buffer affects offline learning performance for all three environments. No uniform exploration (pure exploitation) results in performance staying at random baseline, even after 200K learning steps. Increasing the proportion of uniform exploration yields environment-specific patterns.

CartPole. Small fraction ($\sim 10\%$) of uniform exploration is required to start reaching optimal performance and then plateaus as exploration increases. It reaches peak performance after 5K learning steps; however, for small exploration proportions ($\sim 10\text{-}40\%$), performance degrades as learning progresses above 5K steps.

We hypothesize that even a slight uniform coverage of state-action space helps the agent learn which states to avoid (those where the pole is about to fall), while overestimation of Q-values in underrepresented regions of state-action space or overfitting to the replay buffer may explain the performance degradation during further training. This is consistent with replay buffer and exploitation

policy visualizations in Figure 5.4a and Figure 5.3, which show that exploitation policy covers only a narrow region of the state-action space around the equilibrium point, corresponding to small angles, low angular velocities, and near-zero actions.

Pendulum. Performance, given a small learning budget (~10-20K steps), forms a bell curve peaking at an intermediate exploration proportion (~50%). As learning progresses, higher exploration proportions eventually converge to similar performance levels.

This suggests that Pendulum requires a balance between uniform exploration and exploitation to learn quickly, as high-quality exploitation trajectories provide the precision needed to maintain this unstable equilibrium. As shown in Figure 5.4b, exploitation policy produces transitions densely concentrated in the region close to the upright position, characterized by low angular velocity and high corrective torque. These dense, high quality transitions likely help the agent learn the fine control required for balancing, while uniform transitions prepare it for recovery from states far from equilibrium.

MountainCar. The general pattern is similar to CartPole. However, its performance plateaus only passing ~20% of uniform exploration, and it requires at least 10K learning steps to start reaching optimal performance. It doesn't exhibit performance degradation during further training.

This suggests that MountainCar requires more exploration to find out the way to reach the goal by gaining momentum. It can be explained by the fact that exploitation policy almost never provides negative actions so this part of state-action space is completely uncovered, as depicted in Figure 5.4c. Furthermore, it doesn't fall into a trap of do-nothing policy, likely because both uniform exploration and exploitation cover the goal region (high-position states near the flag, visible as the rightmost part of the state space in Figure 5.4c).

Takeaway. Uniform exploration is consistently beneficial across all environments, but the optimal proportion and required learning budget are environment-dependent, reflecting the specificities of each task. CartPole needs only a small fraction (~10%) and learns quickly, but is sensitive to overtraining. Pendulum requires a balanced mix (~50%) to learn the precise control needed for the unstable equilibrium. MountainCar needs at least ~20% to discover the momentum-building trajectories required to reach the goal.

6.2 Action Noise

As shown in the second row of Figure 6.5, small truncated Gaussian action noise ($\sigma \ll 1$) on exploitation policy actions results in performance at the random baseline for all environments. Increasing the noise level reveals environment-specific responses, which we detail below.

CartPole. Performance rises above random baseline for noise levels with $\sigma \geq 1$ and then plateaus at intermediate performance. These performances are reached at 10K steps, whereas larger learning budget doesn't affect performance.

We hypothesize that exploring around optimal policy is insufficient for CartPole to encounter failure scenarios, except at higher noise levels approaching the uniform action selection. Replay buffer visualization (Figure 5.4a) supports this hypothesis, as action noise increases up to $\sigma = 1$, the state-action space coverage dramatically increases, however further noise level increase doesn't change coverage anymore, likely because random walk policy leads to early episode termination.

Pendulum. Optimal performance is reached only with the noise at $\sigma = 10$; intermediate performance is reached with $\sigma = 100$, but requires at least 50K learning steps.

It suggests that, as with uniform exploration, Pendulum still requires a balance between exploitation and exploration around policy. This is supported by Figure 5.4b, which shows that intermediate noise levels better cover the region around equilibrium. Higher action noise results in a better coverage of

the downward pendulum position for intermediate angular velocities. This can largely help the agent to recover from situations in which it has to swing from downward position.

MountainCar. Performance above the random baseline requires noise at or above $\sigma = 1$. At this level, after more than 20K learning steps, it eventually reaches the goal and yields positive performance. However, higher noise level makes MountainCar immediately plateau at the zero performance.

The corresponding replay buffer visualizations in Figure 5.4c suggest that with small action noise MountainCar, even though it discovers the goal, does not sufficiently discover the momentum-building trajectories required to reach it consistently. This is likely addressed by increasing noise up to $\sigma = 1$. However, with $\sigma > 1$, the random walk policy doesn't discover the goal anymore and rather discovers the valley bottom. Since each action of the car is penalized and the goal is not discovered, it learns a do-nothing policy at zero performance.

Takeaway. Applying truncated Gaussian noise to exploitation policy actions yields moderate performance at intermediate noise levels. Beyond the optimum, further increasing noise approaches a random walk policy, with environment-specific consequences: performance is unaffected for CartPole, slightly degraded for Pendulum, and collapses to zero for MountainCar.

6.3 Branching Noise

The third row of Figure 6.5 shows results for varying levels of branching noise, with 3 branches of depth 3. Each environment displays different dynamics, which we detail below.

CartPole. Performance remains at the random baseline for all learning step budgets and for all levels of noise.

As shown in Figure 5.4a, the corresponding replay buffers do not cover failure and near-failure states at all (corresponding to pole angles beyond 12° , at point the episodes terminates). This is likely because branches of depth 3 are too short to reach episode termination from near-equilibrium states.

Pendulum. Pendulum requires noise at or above $\sigma = 1$ to exceed the random baseline, and reaches optimal performance only at 200K learning steps.

It suggests that with branching noise, exploration remains sufficiently close to optimal trajectories for the agent to learn effective balancing, even at high noise levels. This is consistent with Figure 5.4b, which shows that even high branching noise preserves dense coverage near the upright equilibrium, unlike action noise which disperses transitions more broadly across the state-action space.

MountainCar. Performance shows no clear trend across noise levels, with standard deviation spanning almost the entire reward range. Interestingly, even near-zero noise eventually yields positive performance.

As shown in Figure 5.4c, near-zero branching noise produces transitions concentrated along a single repeated trajectory. We hypothesize that repeatedly observing one coherent trajectory may be more beneficial than seeing fragments of diverse trajectories, as in the pure exploitation buffer. However, this coverage remains insufficient to consistently learn the momentum-building strategies required to reach the goal, explaining the high variance and lack of a clear trend across noise levels.

Takeaway. Branching noise around the exploitation policy provides better coverage of near-optimal regions of the state-action space, proving beneficial for Pendulum and MountainCar. However, it fails entirely for CartPole, as short branches are insufficient to reach episode termination, leaving failure states absent from the replay buffer.

6.4 Comparison of Exploration Strategies

Uniform exploration consistently outperforms noise-based strategies across all environments, providing a useful upper bound on offline learning performance and a way to characterize how exploration-dependent each environment is. However, it requires arbitrary state-space access and does not scale to high-dimensional settings due to the curse of dimensionality, limiting its practical applicability.

Among noise-based strategies, branching noise performs at least as well as action noise for Pendulum and MountainCar, but fails entirely for CartPole. CartPole requires observing episode terminations to learn failure avoidance — short branches never reach termination, leaving failure states absent from the replay buffer and causing OOD errors during evaluation. More generally, noise-based methods leave parts of the state-action space uncovered, causing TD3 algorithm to extrapolate poorly to unseen regions.

One exception favors branching noise over action noise: at large noise levels for MountainCar, high action noise produces a random walk that never discovers the goal, eliminating a critical learning signal. Branching noise, by staying closer to the optimal policy, preserves this signal and maintains positive performance.

A further argument in favor of noise-based strategies in general is that they restrict exploration to reachable regions of the state-action space, avoiding the waste of replay buffer capacity on transitions corresponding to unreachable or physically irrelevant states — a limitation inherent to uniform exploration.

Takeaway. Uniform exploration is the most effective but least realistic strategy. Among noise-based alternatives, branching noise is generally preferable to action noise, except when short branches fail to cover critical states such as CartPole failures.

To test whether OOD extrapolation is the main culprit of noise-based methods, we repeated the experiments using IQL, an offline algorithm specifically designed to handle OOD states.

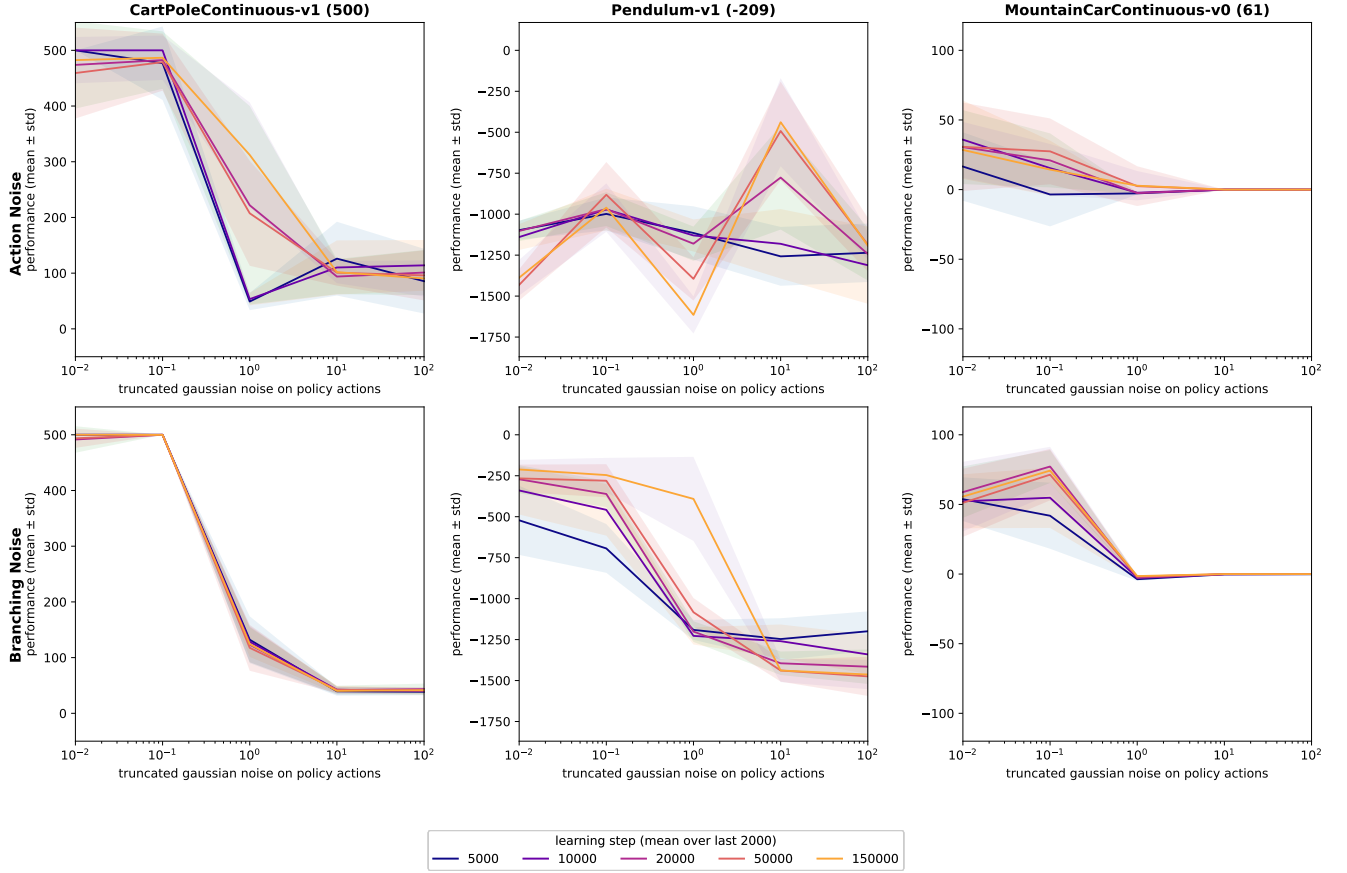


Figure 6.6: Evaluation performance during IQL offline learning for varying replay buffer compositions. Same conventions as Figure 6.5.

6.5 IQL Might Rescue OOD

Figure 6.6 shows experiments on action and branching noise exploration strategies with the same setting as in Figure 6.5, but using IQL algorithm for offline learning.

The results show a trend largely opposite to TD3: small noise $\sigma < 1$ around the optimal policy yields optimal or intermediate performance, while larger noise degrades performance to the random baseline for CartPole and Pendulum, and to zero for MountainCar. An exception is Pendulum, which reaches optimal performance only at $\sigma = 10$ after 50K steps.

This reversal is consistent with IQL’s design: by avoiding querying out-of-distribution actions during policy improvement, IQL can effectively exploit low-noise replay buffers that preserve high-quality transitions near the optimal policy. Conversely, high noise degrades performance because the dataset no longer concentrates transitions in informative regions, and IQL has no mechanism to compensate for poor data quality.

Takeaway. IQL largely reverses the trend observed with TD3: low noise yields good performance while high noise degrades it, confirming that OOD extrapolation is a primary limitation of TD3 in the noise-based exploration setting.

7 Conclusion

This project studied how replay buffer composition affects offline reinforcement learning performance across three environments with different exploration requirements.

Uniform exploration proved consistently beneficial. The optimal exploration proportion and required learning budget are environment-dependent:

- CartPole needs only ~10% and learns quickly but is sensitive to overtraining.
- Pendulum requires a balanced mix (~50%) to acquire the precise control needed for its unstable equilibrium.
- MountainCar needs at least ~20% to discover the momentum-building trajectories required to reach the goal

Despite its effectiveness, uniform exploration requires arbitrary state-space access, does not scale to high dimensions and may sample irrelevant or even unfeasible regions of state-action space.

Noise-based strategies offer a more realistic alternative, but often suffer from incomplete state-action space coverage.

Action noise yields moderate performance at intermediate noise levels, with environment-specific degradation beyond the optimum.

Branching noise provides better coverage of near-optimal regions, proving beneficial for Pendulum and MountainCar, but fails entirely for CartPole where short branches never reach episode termination. Among noise-based strategies, branching noise is generally preferable, except where critical states require longer exploration trajectories.

The choice of offline learning algorithm significantly mediates these effects. IQL largely reverses the trends observed with TD3 under noise-based exploration, corroborating that OOD extrapolation is a primary limitation of noise-based strategies.

A direction for future work is the analysis and visualization of the offline-learned policies themselves, rather than only their evaluation performance. Examining how learned policies cover the state-action space and where they fail would allow direct corroboration of the hypotheses proposed in this work. For instance, whether policies trained on noise-based buffers indeed behave poorly in OOD regions, or whether CartPole policies trained without failure states lack any recovery behavior.

Bibliography

- Denoyer, L., Fuente, A. d. l., Duong, S., Gaya, J.-B., Kamienny, P.-A., & Thompson, D. H. (2021, October 15). *SaLinA: Sequential Learning of Agents*. arXiv. <https://doi.org/10.48550/arXiv.2110.07910>
- Fujimoto, S., & Gu, S. S. (2021, December 3). *A Minimalist Approach to Offline Reinforcement Learning*. arXiv. <https://doi.org/10.48550/arXiv.2106.06860>
- Fujimoto, S., Hoof, H. v., & Meger, D. (2018, October 22). *Addressing Function Approximation Error in Actor-Critic Methods*. arXiv. <https://doi.org/10.48550/arXiv.1802.09477>
- Konda, V., & Tsitsiklis, J. (1999). Actor-Critic Algorithms. *Advances in Neural Information Processing Systems*, 12. <https://proceedings.neurips.cc/paper/1999/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- Kostrikov, I., Nair, A., & Levine, S. (2021, October 12). *Offline Reinforcement Learning with Implicit Q-Learning*. arXiv. <https://doi.org/10.48550/arXiv.2110.06169>
- Kumar, A., Zhou, A., Tucker, G., & Levine, S. (2020, August 19). *Conservative Q-Learning for Offline Reinforcement Learning*. arXiv. <https://doi.org/10.48550/arXiv.2006.04779>
- Lambert, N., Wulfmeier, M., Whitney, W., Byravan, A., Bloesch, M., Dasagi, V., Hertweck, T., & Riedmiller, M. (2022, January 27). *The Challenges of Exploration for Offline Reinforcement Learning*. <https://arxiv.org/abs/2201.11861v2>
- Levine, S., Kumar, A., Tucker, G., & Fu, J. (2020, November 1). *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems*. arXiv. <https://doi.org/10.48550/arXiv.2005.01643>

- Seno, T., & Imai, M. (2022, December 3). *d3rlpy: An Offline Deep Reinforcement Learning Library*. arXiv. <https://doi.org/10.48550/arXiv.2111.03788>
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: an introduction* (Second edition). The MIT Press.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., Kg, A., Perez-Vicente, R., Pierré, A., Schulhoff, S., Tai, J. J., Tan, H., & Younis, O. G. (2024, July 24). *Gymnasium: A Standard Interface for Reinforcement Learning Environments*. <https://arxiv.org/abs/2407.17032v4>
- Wang, D., Li, L., Wei, W., Yu, Q., Hao, J., & Liang, J. (2024). Improving Generalization in Offline Reinforcement Learning via Adversarial Data Splitting. *Proceedings of the 41st International Conference on Machine Learning*, 50822–50839. <https://proceedings.mlr.press/v235/wang24aj.html>

A Appendix

Algorithm: Uniform Sampling

```
# input: environment, empty replay_buffer
while not replay_buffer.full():
    s = env.state_space.uniformly_sample()
    a = env.action_space.uniformly_sample()
    env.state = s
    r, s_prime, terminated = env.step(a)
    transition = (s, a, r, s_prime, terminated)
    replay_buffer.add_transition(transition)
return replay_buffer
# output: full replay_buffer
```

Listing A.1: Uniform Sampling Algorithm Pseudocode

Algorithm: Action Noise

```
# input: environment, policy, noise (sigma), empty replay_buffer
while True:
    # run episodes in the environment
    terminated = False
    env.reset()
    s = env.state
    while not terminated:
        a = add_truncated_gaussian_noise(policy(s), sigma)
        r, s_prime, terminated = env.step(a)
        transition = (s, a, r, s_prime, terminated)
        replay_buffer.add_transition(transition)
        if replay_buffer.full():
            break
        s = s_prime
    if replay_buffer.full():
        break
return replay_buffer
# output: full replay_buffer
```

Listing A.2: Action Noise Algorithm Pseudocode

Algorithm: Branching Noise

```

# input: environment, policy, noise (sigma), number of branches (k)
# depth of branches (d), empty replay_buffer()
while True:
    # run episodes in environment
    terminated = False
    s = env.reset()
    while not terminated:
        # k the number of branched
        for i in range(k):
            env_b = env.copy()
            s_b = env_b.state
            # how depth of each branch
            for j in range(d):
                a_b = add_truncated_gaussian_noise(policy(s_b), sigma)
                r, s_b_prime, terminated_b = env_b.step(a_b)
                transition = (s_branch, a_b, r, s_b_prime, terminated_b)
                replay_buffer.add_transition(transition)
                if replay_buffer.full():
                    break
            s_b = s_prime
        # main path
        a = policy(s)
        r, s_prime, terminated = env.step(a)
        transition = (s, a, r, s_prime, terminated)
        if replay_buffer.full():
            break
        s = s_prime
# output: full replay_buffer

```

Listing A.3: Branching Noise Algorithm Pseudocode

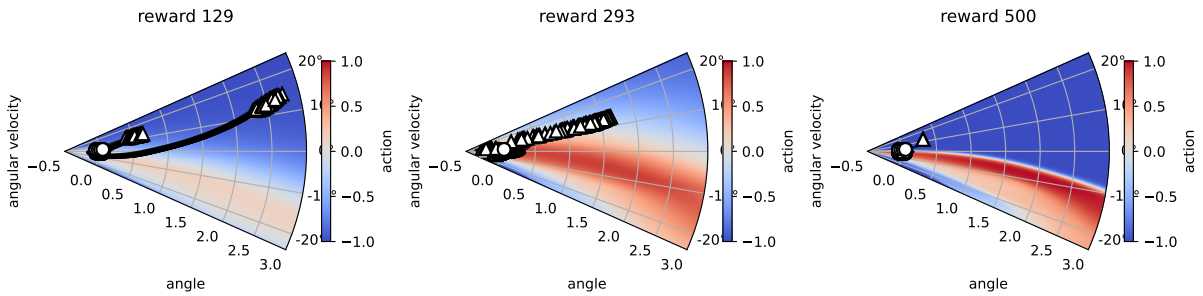


Figure A.7: Online-learned policies for CartPole. Same conventions as Figure 5.3

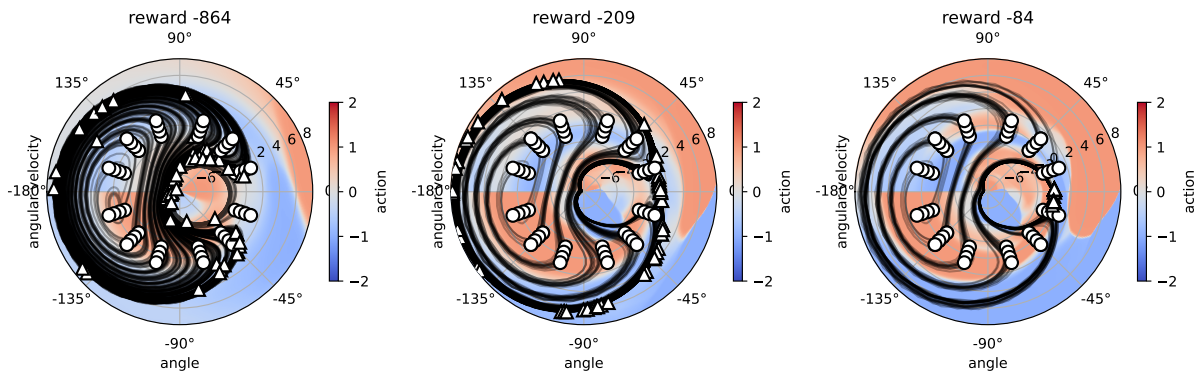


Figure A.8: Online-learned policies for Pendulum. Same conventions as Figure 5.3

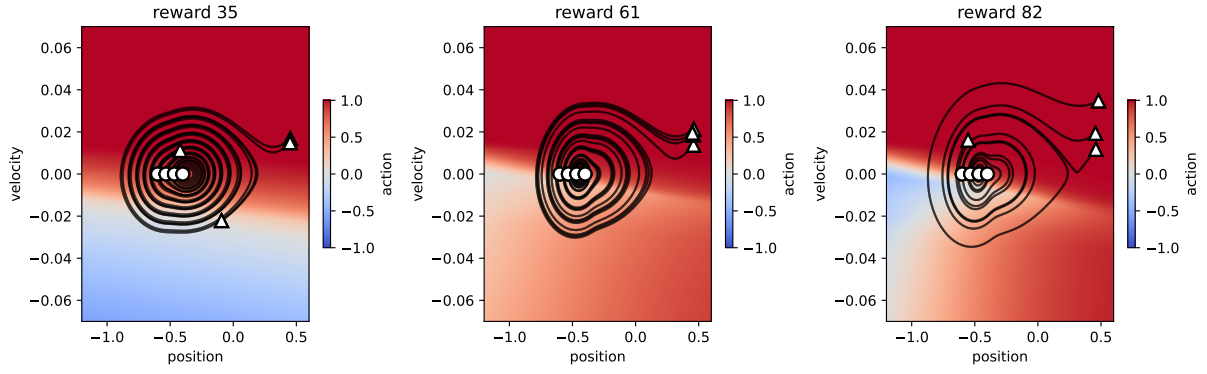


Figure A.9: Online-learned policies for MountainCar. Same conventions as Figure 5.3

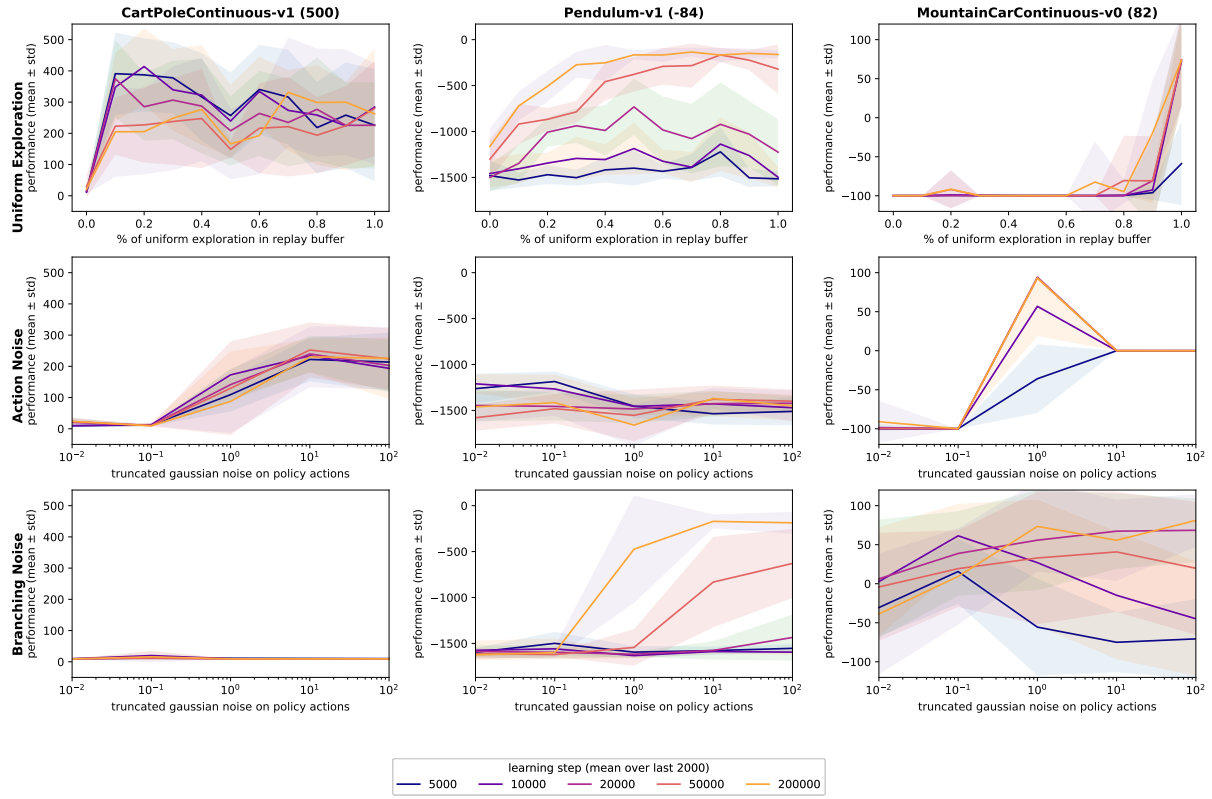


Figure A.10: Evaluation performance during TD3 offline learning for varying replay buffer compositions, using best-performing policies for exploitation. Same conventions as Figure 6.5.

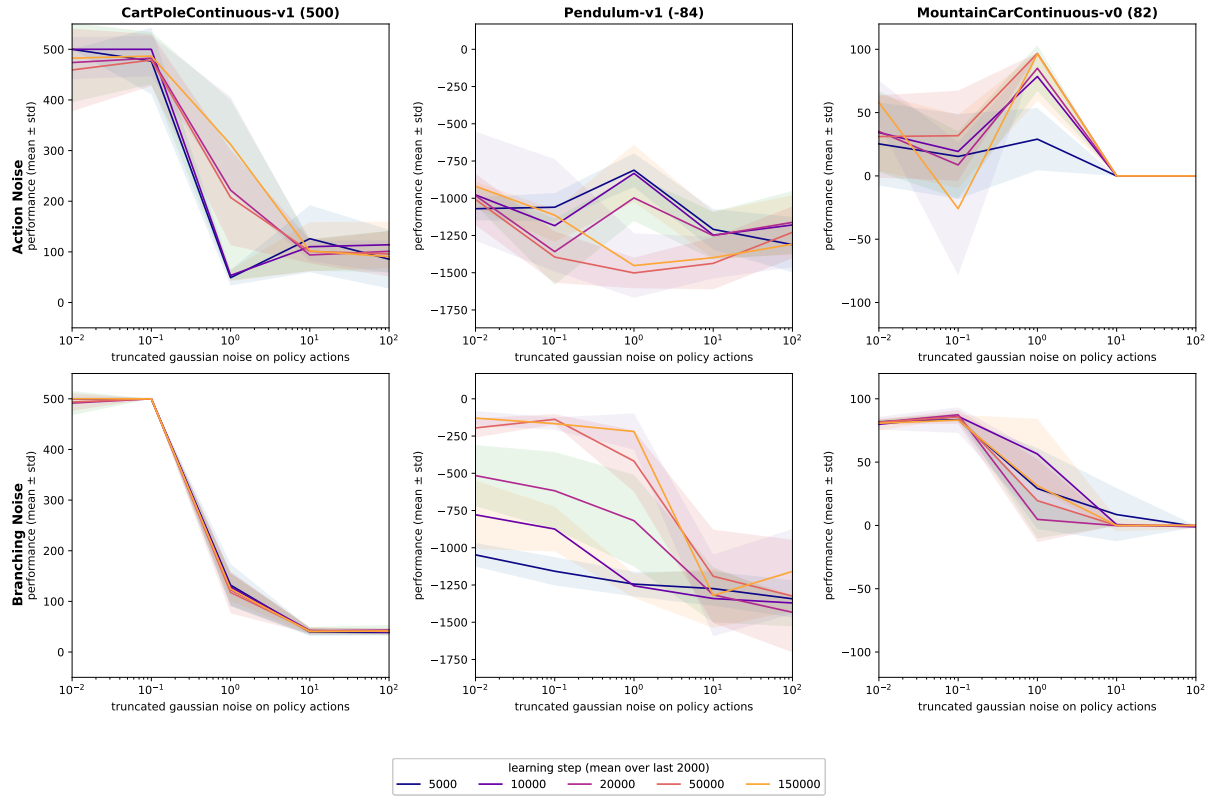


Figure A.11: Evaluation performance during IQL offline learning for varying replay buffer compositions, using best-performing policies for exploitation. Same conventions as Figure 6.5.

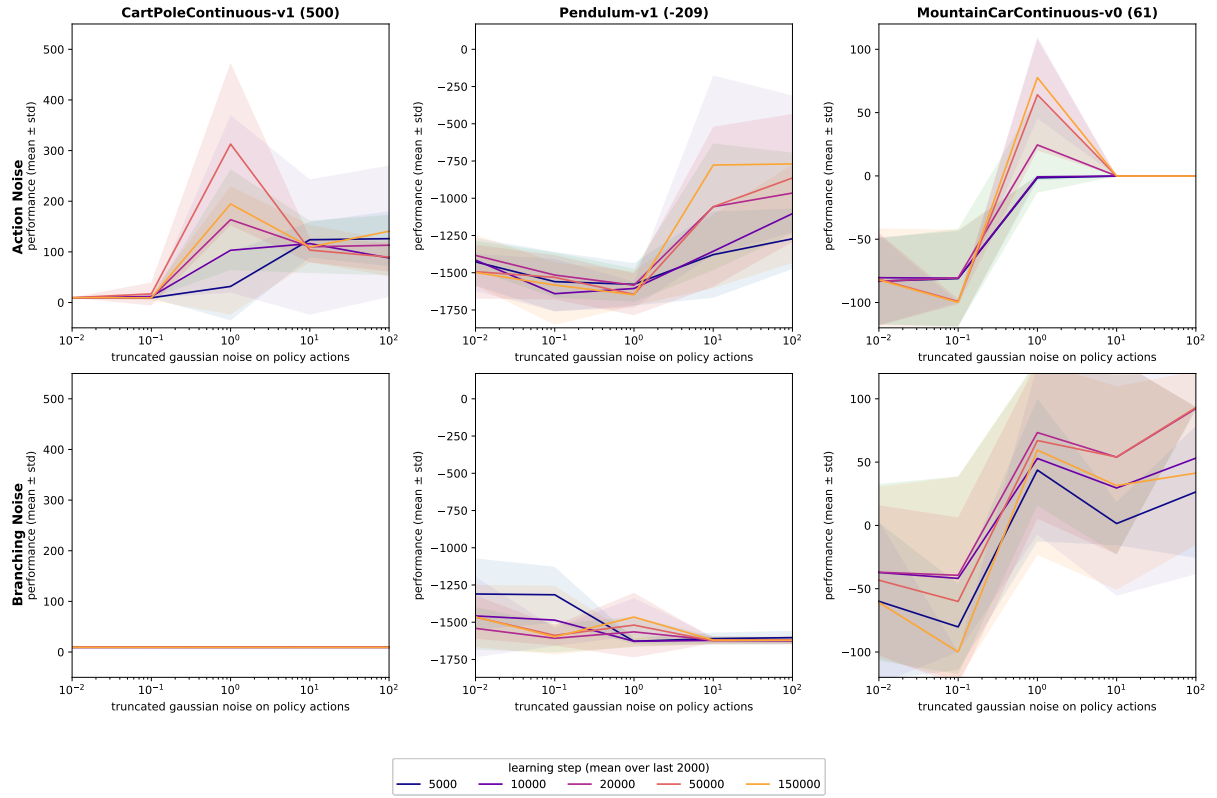


Figure A.12: Evaluation performance during TD3 offline learning for varying replay buffer compositions, using our TD3 implementation with BBRL. Same conventions as Figure 6.5. The obtained results mostly coincide with those obtained with the d3rlpy implementation, except for poorer performance when using branching noise for Pendulum and CartPole. This may be explained by the fact that our implementation does not use target policy noise.

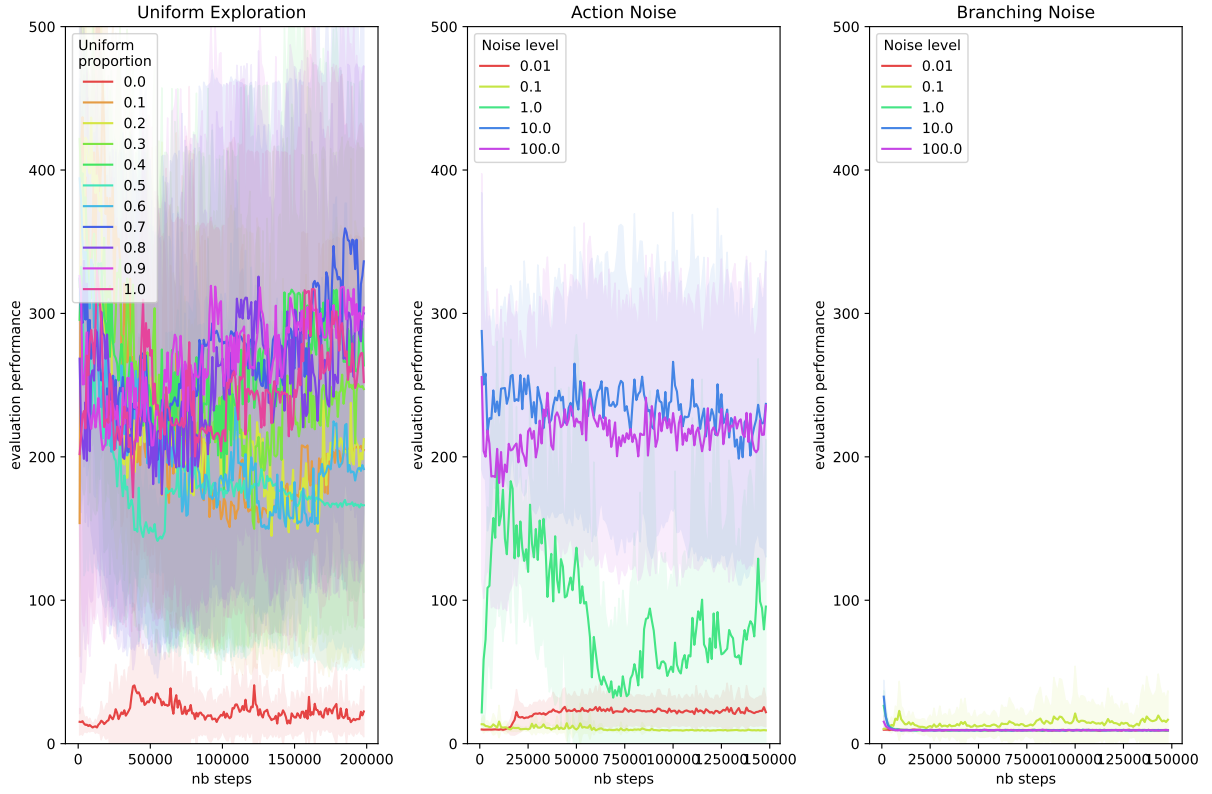


Figure A.13: Learning curves of offline TD3 on CartPole with various replay buffer compositions. Performance was averaged over 10 seeds and 3 evaluations per seed every 1000 gradient steps. Shaded region corresponds to standard deviation. Temporal smoothing with a Gaussian window of size 3 was applied. One step corresponds to one gradient update.

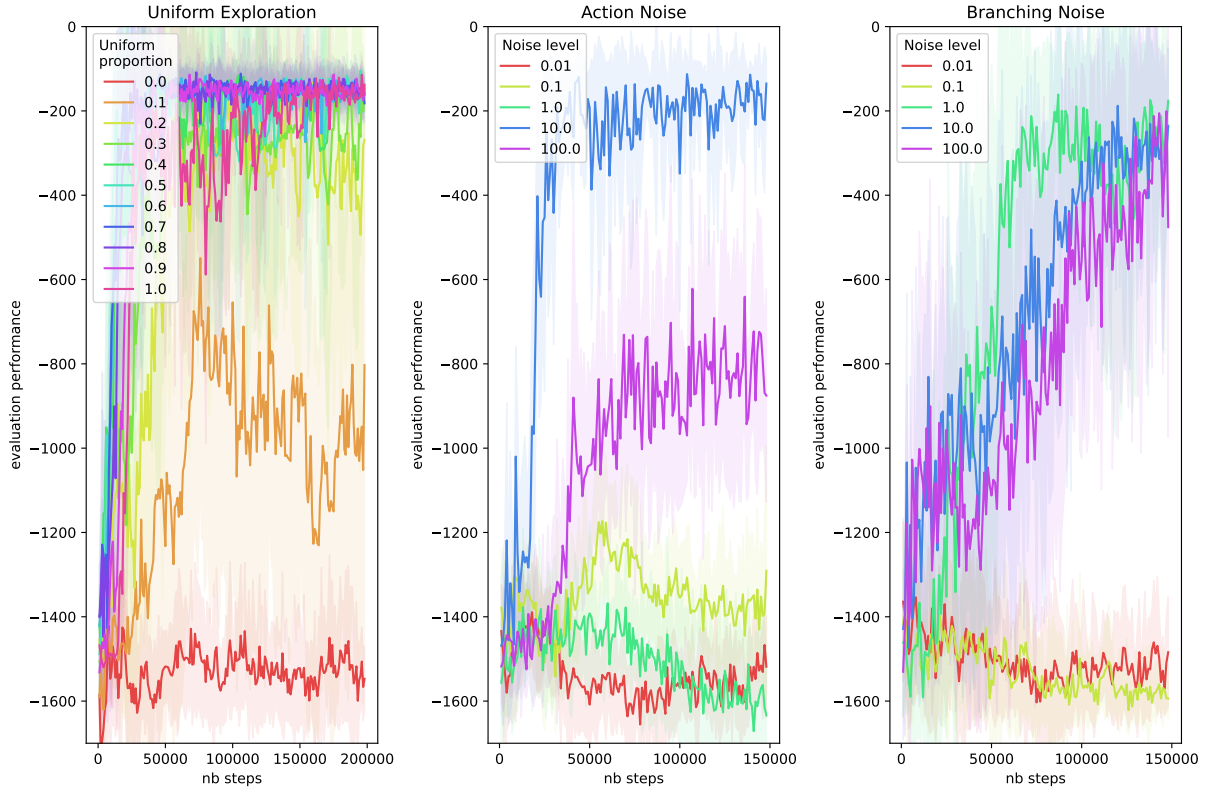


Figure A.14: Learning curves of offline TD3 on Pendulum with various replay buffer compositions. Same conventions as Figure A.13.

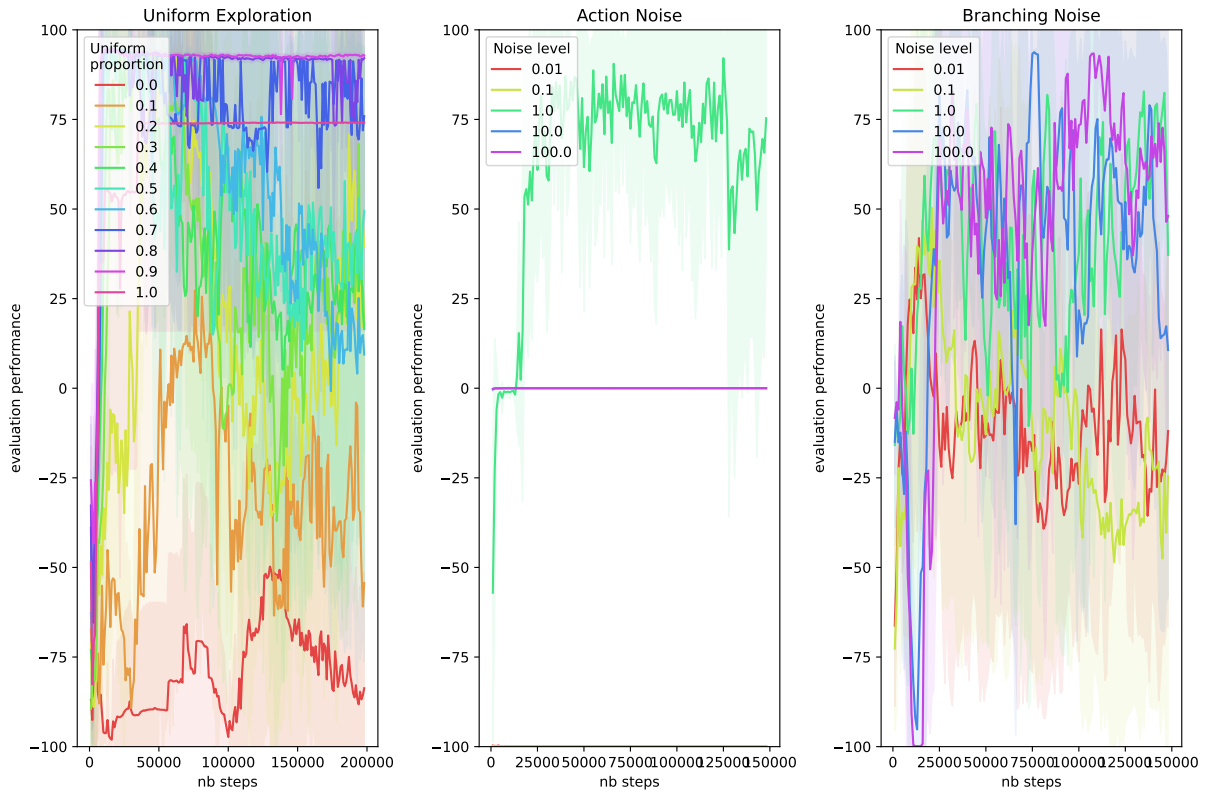


Figure A.15: Learning curves of offline TD3 on MountainCar with various replay buffer compositions. Same conventions as Figure A.13.

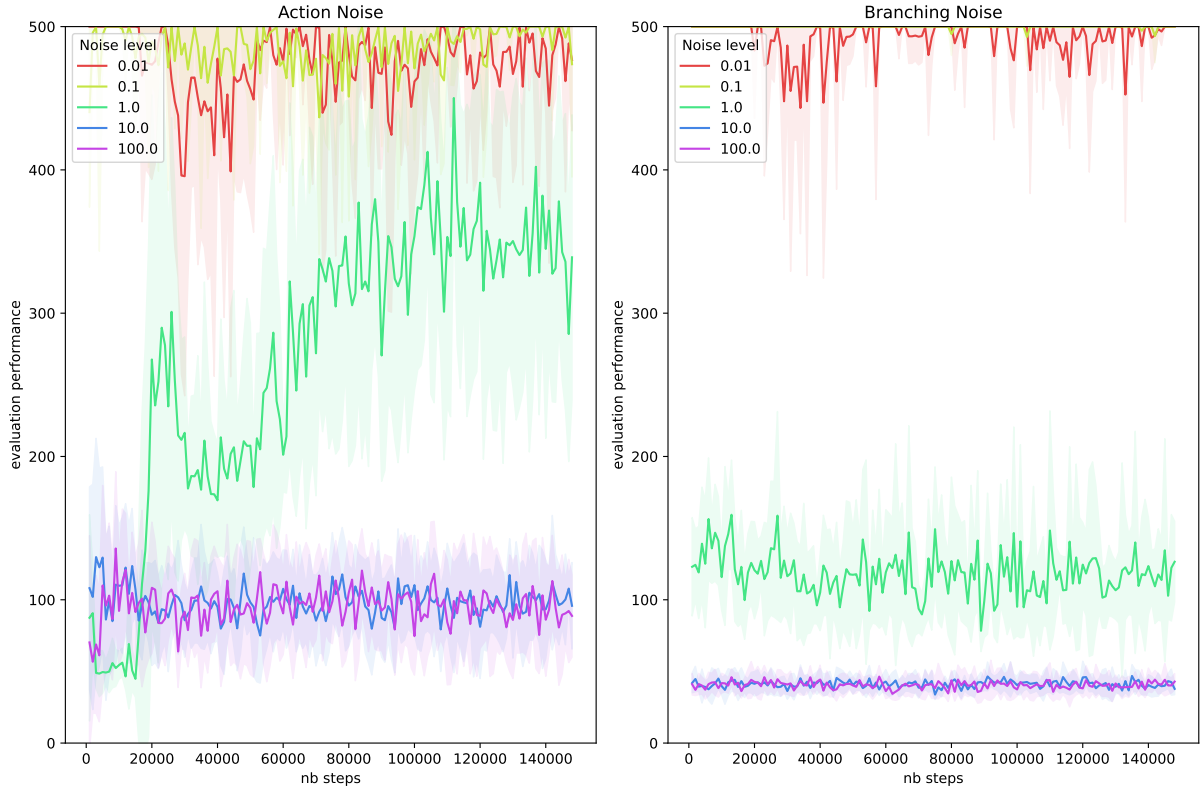


Figure A.16: Learning curves of IQL on CartPole with various replay buffer compositions. Same conventions as Figure A.13.

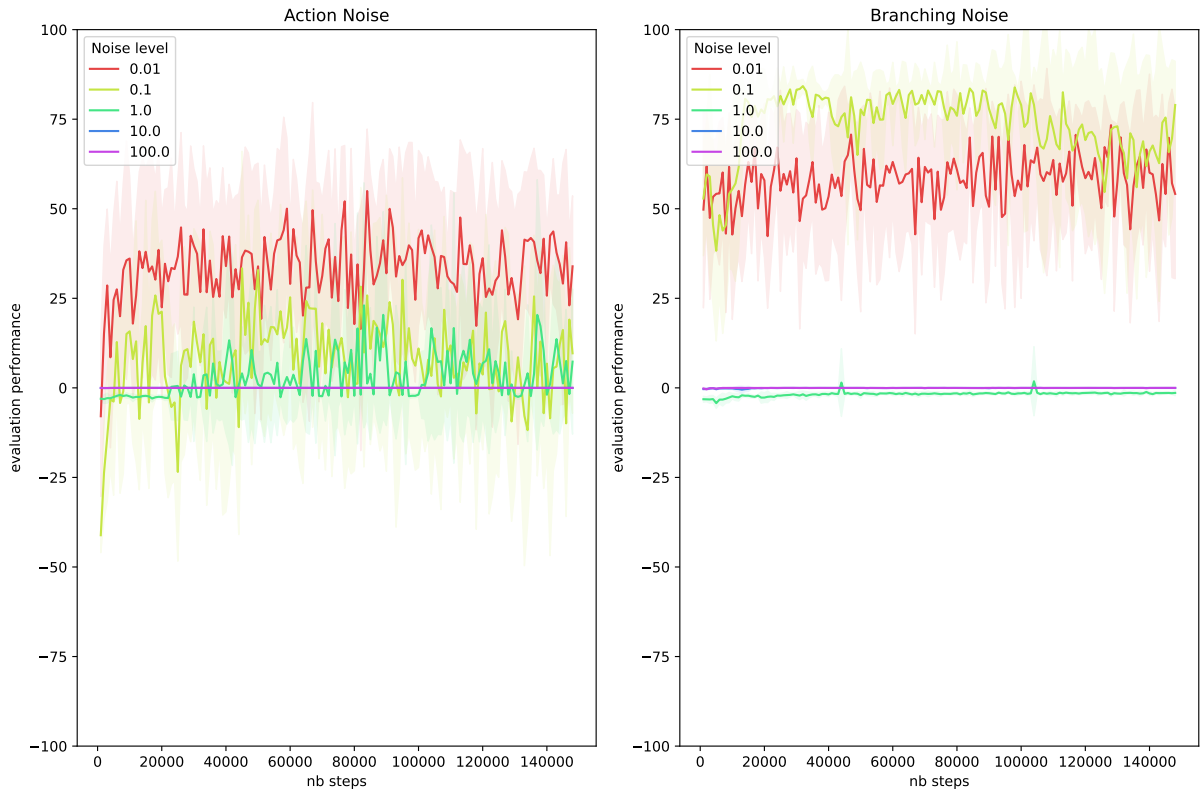


Figure A.17: Learning curves of IQL on MountainCar with various replay buffer compositions. Same conventions as Figure A.13.

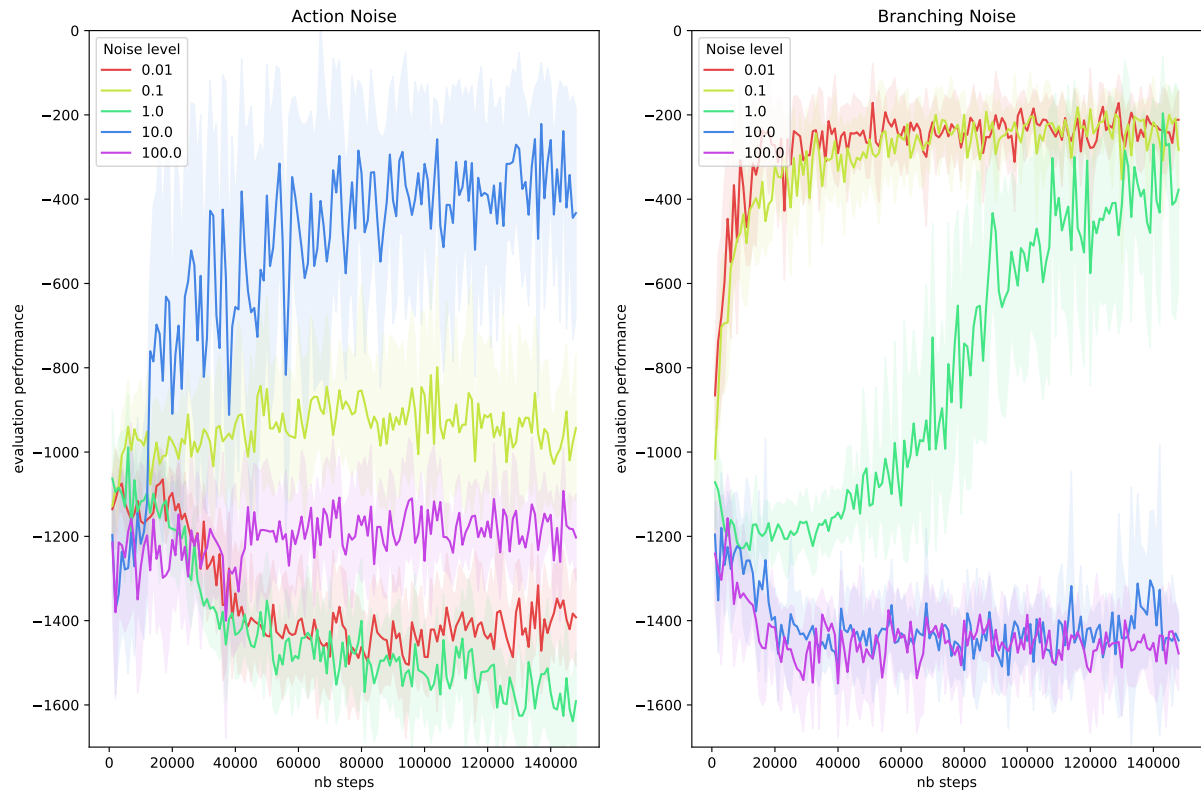


Figure A.18: Learning curves of IQL on Pendulum with various replay buffer compositions. Same conventions as Figure A.13.